

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)

Assessment Tool Weightage: 50%

Dr Dumpala Prasanth

QUESTIONS: 10

Total Marks:50

Annexure A

Constraint - Based Problem-Solving

Duration: 2Hrs

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.
(10 Marks)
2. Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ϵ values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.
(10 Marks)
3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.
(10 Marks)
4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.
(10 Marks)
5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of

deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

(10 Marks)

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

(10 Marks)

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization.

(10 Marks)

8. Analyse an experimental comparison between random action selection and ϵ -greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

(10 Marks)

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

(10 Marks)

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

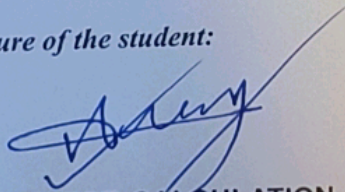
(10 Marks)

Code of Conduct:

(192325060)

I Mohammed Nawaz (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



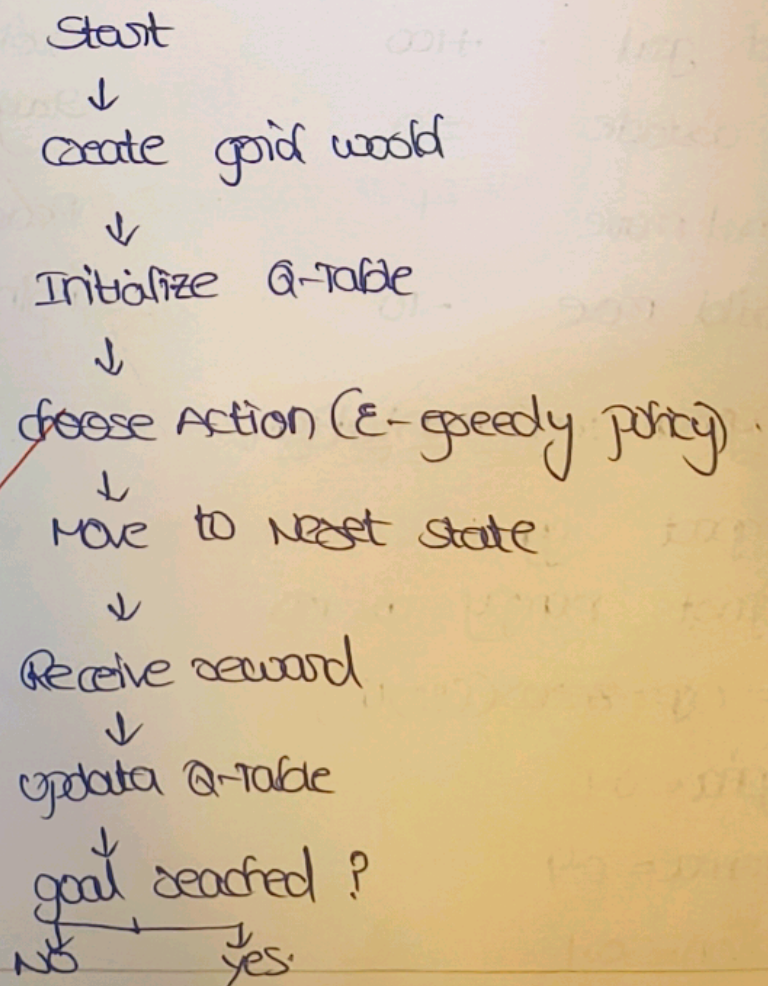
RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Q1) Develop a Python-based Reinforcement learning model using open AI Gym for grid world navigation.

1) Problem understanding:- The objective is to develop a (RL) agent that navigates through a grid world environment. The agent starts from the initial position and learn the optimal path to reach goal while avoiding obstacles. The agent improves its performance with the environment by maximizing the cumulative reward.

2) Flowchart:-



5) Reinforcement Learning Components

<u>Component</u>	<u>Description</u>
Environment	sxs grid world
Agent	RL Agent.
State space	Each grid cell represents one state.
Action space	up, down, left, right.
Goal state	Reach destination safely.
Obstacle state	Agent should avoid these cells

4) Reward function

<u>Situation</u>	<u>Reward</u>
Reach goal	+100
Hit obstacle	-50
normal move	-1
Invalid move	-10

5) Constraints

<u>Constraint</u>	<u>Purpose</u>
Avoid obstacles	collision
Stay within goals	valid
Reach goal	navigation
minimize steps	efficiency.

6) Python Implementation:-

Import gym

Import numpy as np

$Q = np.zeros((25, 4))$

$\alpha = 0.1$

$\gamma = 0.9$

$\epsilon = 0.1$

⑦ working process:-

- 1) create the grid world environment.
- 2) Initialize the Q-table with zero values.
- 3) perform the selected action.
- 4) Receive reward from the environment.
- 5) Receive reward from agent reaches the goal.
- 6) store the learned policy for future navigation.

⑧ Bellman equation:-

$$Q(s,a) = Q(s,a) + \alpha (R + \gamma \max_{a'} Q(s,a') - Q(s,a))$$

where:-

α = learning rate.

γ = Discount factor

R = Reward

s = current state.

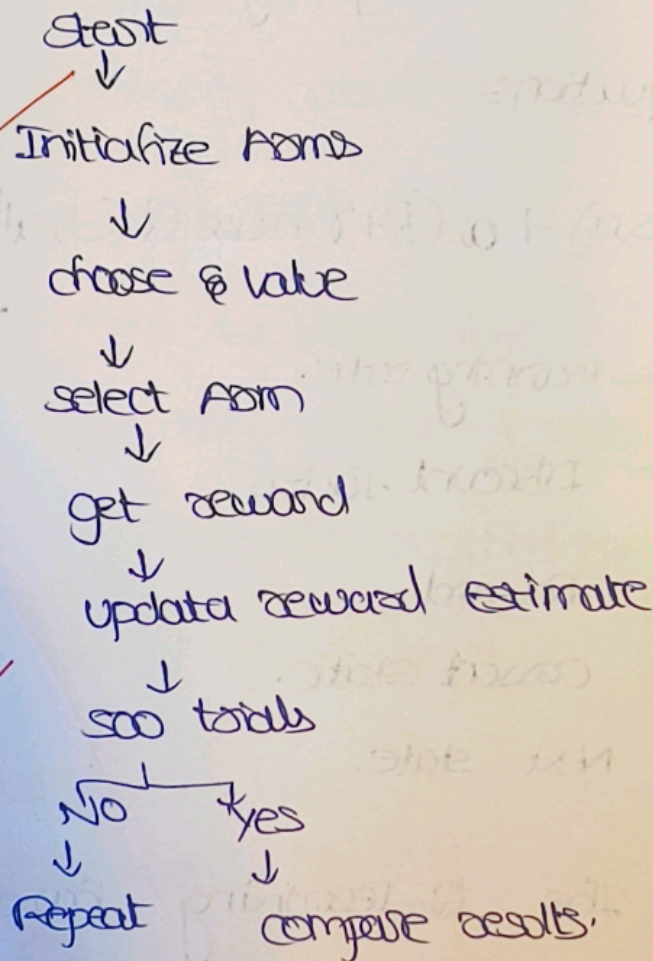
s' = Next state.

⑨ conclusion:- The Q-learning based (RL) model successfully learns an optimal navigation policy.

Q2) Greedy multi-Armed Bandit Algorithm:-

Problem Statement:- The ϵ -Greedy algorithm helps an agent maximize rewards by balancing exploration (trying new arms) & exploitation (choosing the best arm) within 500 trials.

Flowchart:-



Parameters:-

parameter

Arms

Trials

Evalues

value:-

5

500

0.1, 0.3, 0.5

Comparison:-

<u>ϵ</u>	<u>Exploration</u>	<u>Performance</u>
0.1	Low	Best Reward
0.3	Medium	Balanced
0.5	High	Lower reward.

Python syntax:-

```
import numpy as np
epsilon = 0.1
if np.random.rand() < epsilon:
    action = np.random.randint(5)
else:
    action = np.argmax(Q)
```

Syntax meaning:-

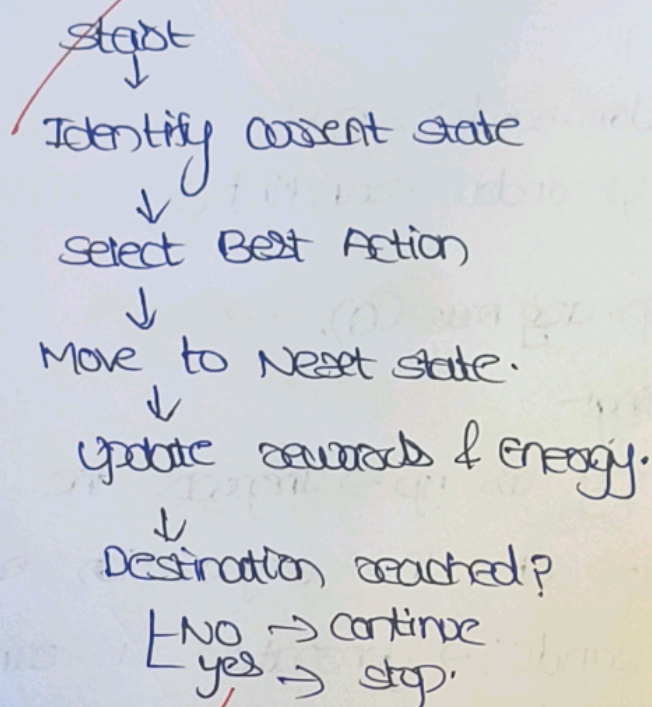
- * `import numpy as np` $\rightarrow$ Imports the numpy library.
- * `epsilon = 0.1` $\rightarrow$ set the exploration rate.
- * `np.random.rand()` $\rightarrow$ generates a random number
- * `np.random.randint(5)` $\rightarrow$ selects a random arm.
- * `np.argmax(Q)` $\rightarrow$ choose the arm with the highest reward.

Conclusion:- The $\epsilon = 0.1$ policy provides the highest cumulative reward by balancing limited exploration with maximum exploitation under the 500 trial constraint.

Q3) Design a markov Decision process (MDP) for an Autonomous Robot.

problem statement: Design an MDP model for an autonomous robot that reaches its destination while reducing energy usage and avoiding unnecessary movement.

Flow chart:



Python syntax:

states = ["start", "path", "Goal"]

actions = ["up", "down", "left", "Right"]

rewards = 100

battery limit = 30.

Syntax meaning:-

States $\rightarrow$ Represents robot positions.

actions $\rightarrow$ possible movement direction.

reward $\rightarrow$ rewards for reaching the goal.

Battery limit $\rightarrow$ maximum energy available for navigation

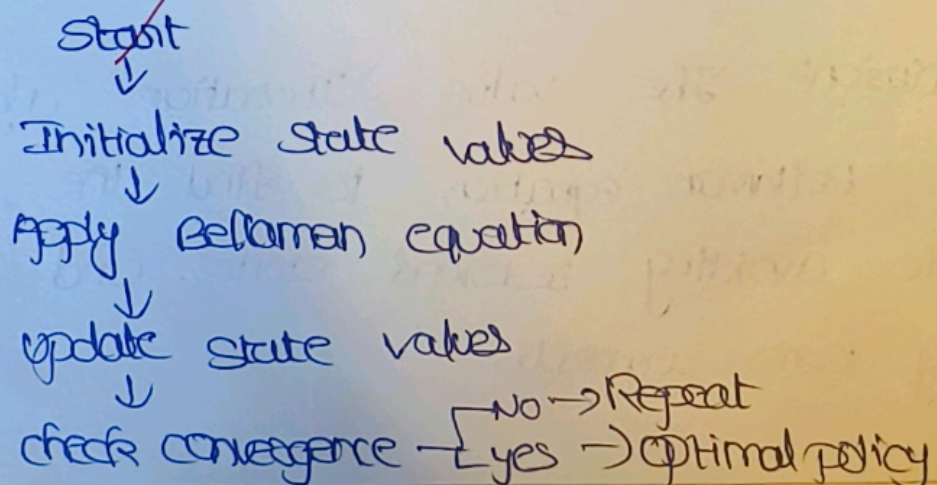
Conclusion:-

The Map enables the robot to choose the optimal path while making energy consumption improving navigation efficiency and battery utilization

Q4:- value iteration Algorithm for a constrained grid environment.

problem statement:- Implement a value iteration algorithm using python (Tensorflow/keras) to find the optimal policy in a grid environment with restricted states.

flow chart:-



Bellman Equation:-

$$V(s) = \max_a [R + \gamma V(s')]]$$

It update each states value and helps select the best action

Python Syntax:-

Import tensorflow as tf

Import numpy as np

gamma = 0.9

values = np.zeros((S,S))

Syntax meaning:-

⇒ Import tensorflow as tf → Imports the TensorFlow library.

⇒ Imports the Numpy library

⇒ gamma = 0.9 → set the discount factor.

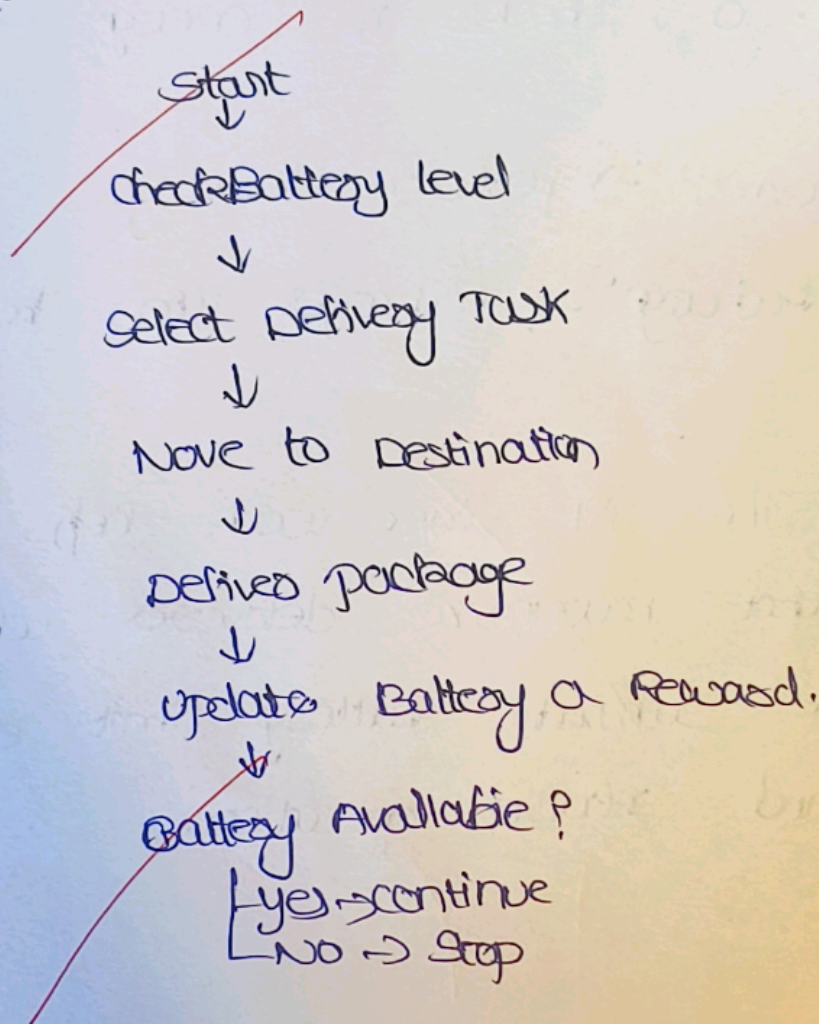
⇒ values = np.zeros((S,S)) → Initializes the values table for the grid.

Conclusion:- The value Iteration algorithm applies the Bellman equation to find the optimal policy while avoiding restricted states and maximizing long term rewards.

5: RL Framework for a warehouse robot with Battery Constraint.

problem statement: Design a (RL) framework for a warehouse robot that completes the maximum no. of deliveries without exceeding a 30-minute battery limit

Flow chart:



Python syntax:

battery = 30.

reward = 50

If battery > 0;

action = "Deliver"

else:-
action = "Recharge"

Syntax meaning:-

Battery = 30 $\Rightarrow$ sets the battery limit to 30 minutes.

reward = 50 $\Rightarrow$ Reward for successful delivery.

If Battery = 0 $\rightarrow$ checks if enough battery is available.

action = "Deliver" $\rightarrow$ performs delivery.

action = "Recharge" $\rightarrow$ Recharges when the battery is low.

Conclusion:- The RL framework helps the warehouse robot maximize deliveries while staying within the 30-minute battery limit, ensuring efficient and reliable operation.